UNIVERSIDAD NACIONAL DE LUJÁN

Departamento de Ciencias Básicas • Lic. en Sistemas

Programación Orientada a Objetos (11[0|2]76)

Clase 05 • **Ejercicio 02** • Ciclo Lectivo 2026

MATERIAL PARA ESTUDIANTES
ENUNCIADO PRÁCTICO

Ejercicio 2: Registro Único de Clientes y Tarjetas con TDD

Contexto

En este ejercicio aplicaremos el ciclo **Test-Driven Development (TDD)** para modelar la integridad referencial y las validaciones de unicidad a lo largo de un grafo de objetos en memoria (`Banco` → `Cliente` → `Tarjeta`), protegiendo el sistema contra la **fuga de encapsulamiento (*aliasing*)**.

Enunciado

Una entidad bancaria (**`Banco`**) administra **Clientes** (`Cliente`) identificados por su DNI. Cada cliente puede poseer múltiples **Tarjetas de Crédito** (`Tarjeta`), identificadas por un número único en toda la institución.

Reglas de negocio e invariantes:

1. **Unicidad de Clientes:** No se pueden registrar dos clientes con el mismo DNI en el banco. Intentar registrar un DNI duplicado debe lanzar `ClienteDuplicadoException` .
2. **Precondición de Asociación:** Solo se pueden asociar tarjetas a clientes previamente registrados en el banco. Si el DNI no existe, debe lanzarse `IllegalArgumentException` .

3. **Unicidad Global de Tarjetas:** Un número de tarjeta debe ser estrictamente único en todo el banco. Si se intenta asociar una tarjeta que ya pertenece a otro cliente (o al mismo cliente), debe rechazarse lanzando `TarjetaDuplicadaException`.
4. **Atomicidad del Registro:** Si una asociación es rechazada por tarjeta duplicada, el cliente no debe incorporar la tarjeta a su tenencia y el banco no debe alterar sus registros.
5. **Protección contra Aliasing:** La consulta de tarjetas (`cliente.getTarjetas()`) no debe permitir modificaciones externas sobre la lista interna del cliente.

Tareas:

1. Diseñe mediante TDD las clases `Tarjeta`, `Cliente`, `Banco` y las excepciones `ClienteDuplicadoException` y `TarjetaDuplicadaException`.
2. Implemente en `Banco` las operaciones públicas:
    - `registrarCliente(String dni, String nombre)`: Da de alta al cliente validando que el DNI no esté repetido.
    - `asociarTarjeta(String dni, String numeroTarjeta)`: Asocia una nueva tarjeta al cliente indicado, validando que el número no haya sido emitido previamente en todo el banco.
    - `buscarCliente(String dni)`: Localiza y retorna al cliente correspondiente o lanza `IllegalArgumentException` si no existe.
3. Escriba pruebas unitarias con JUnit 5 avanzando incrementalmente desde el registro básico hasta el rechazo de tarjetas duplicadas y el blindaje de la colección inmodificable.

Restricciones y Tips

- **Unicidad Global y Eficiencia:** Considere cómo verificar la existencia previa de la tarjeta. ¿Recorrerá el grafo con Streams (`flatMap`) o mantendrá en `Banco` un índice secundario (`Set<String> tarjetasEmitidas`) de búsqueda en tiempo constante $O(1)$?
- **Fuga de Encapsulamiento (*Aliasing*):** Asegúrese de que `Cliente.getTarjetas()` retorne una lista inmodificable con `Collections.unmodifiableList(...)`. Valide con un test que cualquier

intento de hacer `.add(...)` sobre la lista obtenida lance `UnsupportedOperationException`.

- **Testing de Atomicidad:** Cuando un test espere `TarjetaDuplicadaException`, verifique inmediatamente después que el cliente conserve intacta su cantidad original de tarjetas.

Preguntas Conceptuales

Al finalizar el ejercicio, reflexione sobre las siguientes preguntas:

1. ¿Qué ventajas y desventajas en rendimiento ($O(1)$ vs $O(N \times M)$) representa mantener en `Banco` un `Set<String>` secundario con los números de tarjeta emitidos frente a recorrer los clientes con Streams?
2. ¿Por qué exponer colecciones mutables a través de un *getter* rompe el encapsulamiento y cómo previene esto `Collections.unmodifiableList`?
3. ¿Por qué es fundamental que un intento fallido de asociar una tarjeta duplicada o a un cliente inexistente no altere el estado ni del `Cliente` ni del `Banco`?
4. ¿Por qué la creación y asociación de una tarjeta debe canalizarse exclusivamente a través del método de negocio `banco.asociarTarjeta(...)` en lugar de permitir que se agregue directamente sobre `Cliente`?